

One crashed container took your site down. Kubernetes never lets that happen again.

Your badge runs today as one container on one server. If it crashes, you're down until you notice and fix it by hand. Today you meet Kubernetes – the system that watches your containers for you and keeps them running. This session is the mental model: the problem it solves and the four objects you need to read any cluster on sight. No console work yet – next session you deploy onto a real one.

IN PARTNERSHIP WITH



From "why bother" to reading any cluster on sight

IDEA 1 The problem

One container, one server: a crash means downtime, and popularity means more work for you.

crash scale

IDEA 2 Desired state

You declare what you want; Kubernetes continuously makes reality match it.

reconcile

IDEA 3 Four objects

Pod, Node, Deployment, Service – everything this week is built from exactly these.

Pod Deployment
Service

IDEA 4 Namespaces

One shared cluster, walled off per student – the same idea as your VPC on Day 3.

namespace

All four ideas build on each other in order – keep this strip in mind as we go.

Why "one container on one server" doesn't survive contact with reality

A crash is a single point of failure

Your container dies – a bug, an out-of-memory kill, anything. Nothing restarts it. Your badge is down until **you** notice and SSH back in.

Scaling means you, by hand

App gets popular? You'd SSH in, run more containers yourself, and wire up a load balancer in front of them – every time traffic changes.

Kubernetes' one job

Watch your containers continuously and keep reality matching what you asked for – restart what dies, spread traffic across every healthy copy.

Everything from here is Kubernetes automating exactly this – nobody SSHes in at 2am to restart a crashed container again.

You declare desired state. Kubernetes enforces it, forever.

You don't tell Kubernetes **how** to run your app step by step. You describe the desired state – "I want 3 copies of this image, always" – and hand it over. Kubernetes runs a tight loop, forever: check reality, compare it to what you asked for, fix any difference.

```
while true:
    reality    = what's actually running
    desired    = what you asked for
    if reality != desired:
        fix it  # start a Pod, or stop one
```

Like a thermostat

You set the target temperature once. The thermostat keeps checking and correcting – you never manually flip the heater on and off again.

This is "self-healing"

A crashed container isn't restarted by a script **you** wrote. It's this loop, noticing reality fell short and fixing it – usually before anyone notices.

The four objects: Pod, Node, Deployment, Service

Pod SMALLEST UNIT

One running copy of your container. Pods come and go – Kubernetes creates and destroys them constantly.

Node WORKER VM

A machine that runs Pods. A cluster has several – this is an ECS instance under the hood, same as your Day 1 server.

Deployment DESIRED STATE

"Run N copies of this image, keep them healthy."
Creates and replaces Pods for you. This is where scaling happens.

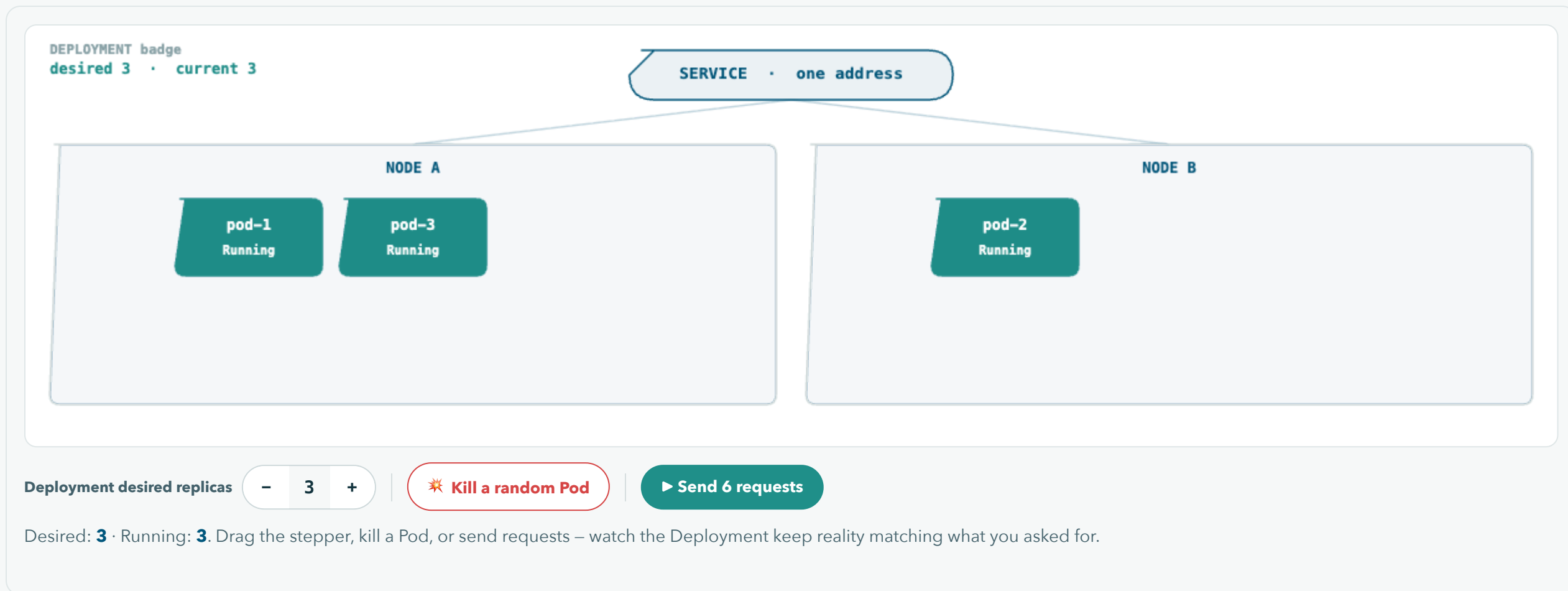
Service ONE ADDRESS

One stable network address in front of your Pods, spreading traffic across them. Pods change constantly; the Service's address never does.



The mental model: **Deployment** = how many and which image · **Pod** = one running copy · **Node** = a machine to run copies on · **Service** = one front door to all of them.

Kill a Pod. Scale it. Watch Kubernetes fix it before you can.



Namespaces: the same trick your VPC pulled on Day 3

One shared cluster

The whole class deploys onto a single ACK cluster your instructor already provisioned. You are not creating a cluster today – it's already running.

Namespace YOUR OWN AREA

A namespace walls off a slice of the cluster – the same way your VPC walled off your own network on Day 3. Every student creates their own, so nobody's Deployment collides with anyone else's.

```
ACK cluster                                (shared by the whole
class)
├─ namespace: sara                        (Sara's Pods live
here)
├─ namespace: turki                      (Turki's Pods live
here)
└─ namespace: you                        ← you create this
next session
```

Every step next session happens in the Alibaba ACK console UI – no terminal, no `kubectl`.

Name the **ONE** object responsible for each – cold, from the definitions

1. One running copy of your container. → _____
2. Keeps N copies running, replacing any that die. → _____
3. The worker machine that physically runs your Pods. → _____
4. One stable address that spreads traffic across the healthy copies. → _____
5. What you create so your Deployment doesn't collide with a classmate's. → _____

Hint: exactly one answer per line, all five terms from this session – no term used twice.

You can read a Kubernetes cluster and explain why it exists

The problem, named

One container on one server can't heal itself or scale itself – that's the whole reason Kubernetes exists.

Four objects, cold

Pod, Node, Deployment, Service – you can say what each one does without looking it up.

After the break

You'll create your own namespace and deploy a real workload onto the class's shared ACK cluster – then scale it and kill it on purpose.

 Kahoot: Day 4 · Session 1 – the Kubernetes problem, desired state, the four objects, and namespaces.

Time to actually run this on a real cluster.

One shared ACK cluster is already running – your instructor provisioned it. You'll create your own namespace, describe a Deployment and a Service in YAML, and watch real Pods come to life, scale up, and heal themselves – all from the Alibaba console.

nginxdemos/hello: a demo image that proves load-balancing on screen

nginxdemos/hello is a small public demo web server. Its homepage prints the hostname of the exact Pod that answered your request. That's not decoration – it's how you'll **see** multiple Pods and load-balancing working, instead of just trusting they are.

Your badge stays exactly where it is, on your Day 1 server. This is a separate, hands-on exercise in Kubernetes patterns you'll recognize on any real job.

Refresh, and watch it change

Load the page, refresh a few times – the "Server address" / "Server name" line flips between your different Pods. Same Service, different Pod each time.

Why a public demo image

No Dockerfile, no registry login, nothing to build. One image name, and the proof is on the screen in one refresh.

Service types: keep it inside the cluster, or expose it to the world

TYPE	REACHABLE FROM	WHEN YOU'D USE IT
ClusterIP	Inside the cluster only	The default – an internal service other Pods call.
LoadBalancer	The public internet	Alibaba automatically wires up a real load balancer in front of your Pods – today's type.

💡 This is the same SLB concept from Day 3's "recognize, don't configure" slide – except today **type: LoadBalancer** in your YAML is the entire configuration. Kubernetes does the rest.

Fill in the blanks so this runs nginxdemos/hello, 3 copies, on port 80

```
# deployment.yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: hello
  namespace: _____ # yours, from Milestone 8
spec:
  replicas: _____ # how many copies?
  selector:
    matchLabels:
      app: hello
  template:
    metadata:
      labels:
        app: hello # the Service looks for this
    spec:
      containers:
        - name: hello
          image: nginxdemos/hello
          ports:
            - containerPort: 80
```

↓ deployment.yaml

↓ service.yaml

```
# service.yaml
apiVersion: v1
kind: Service
metadata:
  name: hello
  namespace: _____ # same as above
spec:
  type: _____ # public address?
  selector:
    app: _____ # match the Pod label
  ports:
    - port: 80
      targetPort: 80
```

Hint: **replicas** is "how many copies" – pick 3. The Service's **selector** must match the Pod's **labels** (both say **app: hello**). **namespace** must match in both files and the one you create in Milestone 8.

All from the Alibaba console – no terminal, no kubectl

- 1 Namespaces and Quotas → Create Namespace → name it uniquely → OK. Use this exact name as **namespace:** in both YAML files above.
- 2 Deployments → Create from YAML → paste your completed **deployment.yaml**.
- 3 Network → Services → Create from YAML → paste your completed **service.yaml**.
- 4 Pods – set the Namespace selector to yours, confirm 3 **hello** Pods are Running. In Network → Services, copy the Service's external address.
- 5 Visit that address in a browser. Refresh 4-5 times – watch the "Server address" line change as the Service load-balances.
- 6 Scale it. In Deployments (namespace = yours), find **hello** → Scale → 5 replicas. Watch two new Pods appear.
- 7 Self-heal. In Pods (namespace = yours), click : on one Pod → Delete → confirm. Watch a fresh one appear within seconds.

- ☐ Your own namespace exists, referenced by both YAML files · **hello** runs as multiple Pods behind the Service's external address · refreshing shows different Pod hostnames · scaled to 5 replicas · deleted a Pod and watched it get recreated.

You can deploy, scale, and self-heal a real workload on Kubernetes

This morning

The problem Kubernetes solves, desired state, the four objects, and namespaces.

This afternoon

Your own namespace, a Deployment and Service on a real shared ACK cluster – scaled and self-healed by hand.

Tomorrow

Branches, pull requests, a build check on your pipeline, and the capstone – documenting the whole week's deployment as your own.

 Kahoot: Day 4 · Session 2 – Deployments, Services, Service types, scaling, and self-healing.

IN PARTNERSHIP WITH

